

Every Kernel Is a Join: Automatic Multi-GPU Parallelism for AI Computations in Einsummable

Zhimin Ding, Chen-Kuan Liao, Chima Adiole, Brianna Barrow, Fangzhou Du, Yu Hsiao, Ge Huang, Yicheng Jin, Ismail Syed, Chris Jermaine

Rice University
Houston, USA

{zd21,bl86,ca31,bmb15,fd30,rh120,gh31,yj67,is38,cmj4}@rice.edu

ABSTRACT

We present Einsummable, a prototype system that accepts a PyTorch-like description of an AI computation and automatically distributes it across a multi-GPU server. Einsummable models every operation as a relational join followed by an aggregation over tensor relations, in which the tuples contain sub-tensors. Each operation exposes its possible decompositions through *join-agg specs*. An optimizer selects decompositions across the whole computation to minimize a communication-cost proxy. Because it searches decompositions rather than a menu of strategies, Einsummable discovers plans that mesh-based auto-parallelizers cannot. Each decomposed operation is implemented by synthesizing an *exchange program*, which is a topology-aware generalization of Volcano’s exchange operator. Despite being fully automatic, Einsummable can outperform custom-designed implementations. For example, on LLaMA transformer blocks on an eight-GPU A100 server, Einsummable achieves a geometric-mean runtime of 8.97 ms, versus 13.65 ms for hand-tuned PyTorch and 14.87 ms for vLLM.

PVLDB Reference Format:

Zhimin Ding, Chen-Kuan Liao, Chima Adiole, Brianna Barrow, Fangzhou Du, Yu Hsiao, Ge Huang, Yicheng Jin, Ismail Syed, Chris Jermaine. Every Kernel Is a Join: Automatic Multi-GPU Parallelism for AI Computations in Einsummable. PVLDB, 20(1): XXX-XXX, 2027. doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://anonymous.4open.science/r/Einsummable-CE45>.

1 INTRODUCTION

Determining how to shard an AI computation across the GPUs of a multi-GPU server, so that adding GPUs actually reduces the time required to run the computation, is one of the central problems in systems-for-AI [20, 32, 42]. The difficulty is that distributing the operations that make up an AI computation (such as dot-product attention [36] or matrix multiplication) can incur significant communication overhead. This overhead can be so significant that it is easily possible to distribute a computation across a set of GPUs and find that adding additional GPUs *increases* the running time.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

In this paper, we describe a prototype AI system called Einsummable that accepts as input a PyTorch-like [27] description of an AI computation and automatically distributes it across the GPUs of a multi-GPU server, with no human expertise required. Einsummable automatically optimizes the *intra-operator parallelism* [42] of the computation. Each individual operation is decomposed so that it runs on all GPUs at once, ideally achieving a speedup close to linear in the number of GPUs. “Intra-operator parallelism” is a general class of parallel decompositions that includes data parallelism, tensor (model) parallelism [32, 39], sequence parallelism [22], sharded data parallelism [28], and so on. Crucially, because Einsummable searches a space of decompositions rather than a menu of named strategies, it is not limited to parallelization schemes that have already been invented.¹ Optimizing for intra-operator parallelism is fundamentally a database problem. Intra-operator parallelism was the foundation of the shared-nothing parallel database systems of the 1980s and 1990s [10], and its automatic orchestration was the centerpiece of Volcano’s exchange operator [12]. Einsummable brings key ideas from database systems to bear on AI workloads.

Einsummable leverages the fact that every operation in a modern AI computation (dot-product attention, softmax, layer normalization, element-wise operations such as the ReLU activation, matrix multiplication, and so on) can be modeled as a relational join followed by a relational aggregation over so-called *tensor relations*, in the spirit of the tensor-relational model of computation [41]. A “tensor relation” is a set of (sub_tensor_key, sub_tensor) pairs used to represent a tensor. For example, a distributed matrix multiply is precisely a join on the shared inner index followed by a SUM aggregation. In the degenerate, purely relational case, the sub_tensor_key identifies a single cell in the tensor and the sub_tensor is a scalar value. However, no AI system that manipulates individual scalar values can be competitive for modern, dense AI computations. A competitive system must instead invoke efficient *kernels*, routines (such as a cuBLAS matrix multiply or FlashAttention [7]) that run on an accelerator (GPU, TPU, etc.) over sub-tensors.

Thus, the question addressed by Einsummable’s design is: How to decompose each AI operation in a larger AI computation into joins and aggregations over tensor relations such that (a) each “joined” set of tuples represents enough work that a high-performance kernel can be applied effectively, and (b) the communication overhead introduced by the decomposition is minimized? Einsummable brings

¹We do not consider multi-server computations, where the slow server-to-server interconnect means that throughput-enhancing methods such as pipeline parallelism [15] are more practical. A modern server can present 72 GPUs in a single NVLink domain (e.g., NVIDIA’s GB200 NVL72) so single machines cover many use cases.

two classical database ideas to the problem of multi-GPU AI computation: the separation of a logical specification from its physical implementation, and cost-based optimization.

Existing auto-parallelizing systems such as FlexFlow, GSPMD, and Alpa [20, 39, 42] search over per-operator sharding annotations drawn from a fixed vocabulary, in which tensor axes are mapped to a logical device mesh. More recent systems such as nnScaler [25] let each operator declare which of its dimensions may be partitioned, but every dimension is still partitioned independent of the data. Unlike these systems, in Einsummable, each operation defines its own space of relational decompositions. The added expressivity is crucial. The classical 3D matrix-multiplication algorithm, for example, shards all three loops of a matrix multiply simultaneously, and therefore cannot be expressed over a two-dimensional device mesh. Or, consider dot-product attention over packed sequences: the sequence dimension may be split only at the boundaries between sequences, positions that depend on the data rather than on the tensor’s shape. And under grouped-query attention, the query-heads dimension cannot be sharded independently, but inherits its sharding from the key/value heads. These constraints are properties of the operator and its input, not of the mesh.

Einsummable uses a three-part solution to the problem of automated decomposition. First, every operation available to an Einsummable programmer (such as dot-product attention or matrix multiplication) must export the ability to produce what we call *join-agg specs*. Each join-agg spec describes one way to decompose the operation’s input tensors into tensor relations so that exactly p tuples are produced by the join that is used to implement the operation, where p is the number of processors; by varying its requests, the system enumerates the operation’s decompositions.

Second, because the communication required to move the tuples of a tensor relation from GPU to GPU and to convert one tensor-relational representation into another can be very costly, Einsummable uses a dynamic programming algorithm to automatically choose the most effective set of decompositions for the overall computation, minimizing an estimate of the total number of bytes that must be communicated.

Finally, Einsummable physically realizes the decomposed computation using *exchange programs*. An exchange program is a topology-aware generalization of Volcano’s exchange operator [12], in which all of the aggregation, repartitioning, and replication between producing and consuming operations is expressed in a domain-specific language. Einsummable synthesizes an exchange program for each operation, guided by a simulator of the actual hardware topology, producing a directed acyclic graph of kernel invocations and data-movement operations, which is compiled into a CUDA graph and executed on the GPU server. Notably, Einsummable never invokes a canned communication pattern such as an NCCL all-reduce. Every communication and aggregation pattern is special-purpose, derived during compilation for the specific data movement required.

In summary, this paper makes the following contributions:

- We show that the operations making up modern AI computations can be modeled as joins followed by aggregations over *tensor relations*, giving a single relational framework that captures data, tensor, sequence, and other intra-operator parallelism strategies as special cases (Section 3).

- We introduce *join-agg specs* through which each operation exposes its valid decompositions (Section 3).
- We present a dynamic-programming optimizer that selects decompositions for an entire computation so as to minimize total communication (Section 4).
- We introduce *exchange programs* through which each decomposed operation is converted to a directed acyclic graph of kernel invocations and data movements, replacing canned collective libraries with communication synthesized per computation (Section 5).
- Experimental results show that Einsummable automatically discovers and implements decompositions whose performance meets and sometimes exceeds that of mature systems such as JAX [4] and PyTorch, even expertly tuned. (Section 6).

2 EINSUMMABLE OVERVIEW

At the highest level, Einsummable allows a programmer to create highly-optimized AI computations in a few lines of code. Einsummable computations run in parallel on multiple AI accelerators without the programmer worrying about how the parallelism happens. In this section, we give a simple Einsummable example from a programmer’s view, and use it to give an overview of the compilation process. Consider a two-layer feed-forward network (FFN), $Y = \text{ReLU}(XW_1)W_2$, where

$$X \in \mathbb{R}^{\text{tokens} \times d_{\text{model}}}, W_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, W_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}},$$

and consequently

$$Y \in \mathbb{R}^{\text{tokens} \times d_{\text{model}}}.$$

A programmer would construct this computation in Einsummable as follows:

```
import einsummable as es

graph = es.createGraph()

x = graph.insertTensor(
    es.tensorDesc([tokens, d_model], dtype))
w1 = graph.insertTensor(
    es.tensorDesc([d_model, d_ff], dtype))
w2 = graph.insertTensor(
    es.tensorDesc([d_ff, d_model], dtype))

act = graph.matmul(x, w1)
hidden = graph.unary(act, "relu", 0.0)
y = graph.matmul(hidden, w2)

graph.save(y)
es.compileGraph(graph)
```

The two calls to `matmul` and the call to `unary` add abstract operators to the graph (abstract operators are defined formally in Definition 3.3 of Section 3). The call to `graph.save` identifies an output that must remain available after execution. The call to `compileGraph` asks Einsummable to map the abstract computation onto hardware. By default, Einsummable queries the local CUDA runtime and compiles for every visible GPU; the programmer may instead name a subset of devices, as in `es.compileGraph(graph,`

device=[0, 1, 2, 3]). The program contains no device assignment for an individual operator, no sharding annotations on an individual tensor, and no explicit communication operations. The number of devices named in this call is the processor count p over which all join-agg specs are formed. Join-agg specs are defined in Definition 3.6 of Section 3. Every abstract operator can be asked to produce join-agg specs, which are formal specifications for the tensor-relational decompositions that can be used to implement the operator. During a logical optimization phase, Einsummable considers the join-agg specs available for each abstract operator and chooses compatible decompositions across the entire FFN so as to minimize communication cost, as described in Section 4.

Once a logical plan has been produced, the compilation process then produces a data-movement schedule among the chosen devices linking each abstract operator (specified as an *exchange program*, described in Section 5.2). The default, `schedule_mode="naive"`, uses a naive exchange program. Setting `schedule_mode="ai"` asks an optimizer, guided by a hardware-topology simulator, to search for a lower-cost schedule of the same decomposed operators (Section 5.4). After compilation, the programmer loads the input tensors (as NumPy arrays), executes the graph, and offloads the result.

From the programmer’s perspective, Einsummable resembles a conventional tensor graph API. However, unlike other systems, each operation denotes an abstract operator with many possible join-agg implementations, and compilation selects devices, decompositions, and a data-movement schedule without programmer involvement. The remainder of the paper describes the machinery beneath the API, such as the tensor and join-agg formalism (Section 3), the logical optimizer that chooses decompositions (Section 4), and the physical optimizer that synthesizes exchange programs (Section 5).

3 TENSORS AND OPERATORS

In this section, we describe the formalisms that underlie our Einsummable implementation.

3.1 Tensors in Einsummable

First, we define the notion of *tensors* and *tensor types*.

Definition 3.1 (Tensor type; tensor). A *tensor type* $\mathcal{T}$ is a set of functions from index vectors to real numbers, and is specified by three quantities: a rank $\mathcal{T}_r$ (a non-negative integer; “r” stands for “rank”, and matrices are rank-2 tensors), an *upper-bound* vector $\mathcal{T}_u$, and a *lower-bound* vector $\mathcal{T}_l$, where the bound vectors are integer vectors of length $\mathcal{T}_r$ satisfying $\mathcal{T}_l[j] \leq \mathcal{T}_u[j]$ for each j . Define $\mathcal{I}(\mathcal{T})$ to be the set

$$\{\mathcal{T}_l[0], \dots, \mathcal{T}_u[0] - 1\} \times \{\mathcal{T}_l[1], \dots, \mathcal{T}_u[1] - 1\} \\ \times \dots \times \{\mathcal{T}_l[\mathcal{T}_r - 1], \dots, \mathcal{T}_u[\mathcal{T}_r - 1] - 1\}.$$

This is the set of all indices or keys that obey the upper and lower bounds of $\mathcal{T}$ (lower bounds are inclusive, upper bounds are exclusive). $\mathcal{T}$ is then the set of all functions from $\mathcal{I}(\mathcal{T})$ to the set of real numbers, and a *tensor* (denoted with an upper-case bold letter such as $\mathbf{U}$) having tensor type $\mathcal{T}$ is simply one of the functions in $\mathcal{T}$. That is, we say $\mathbf{U}$ has tensor type $\mathcal{T}$ if $\mathbf{U} \in \mathcal{T}$.

Zero-bounded tensors are those whose tensor type has a lower-bound vector of all zeros. Einsummable users manipulate zero-bounded tensors, but internally Einsummable tensor types may have non-zero lower bounds, as it allows the system to describe/manipulate sub-tensors more easily. For human interpretability, zero-bounded tensor types are often specified using named indices. A named index is a textual string (such as `num_rows` or `num_cols`) that corresponds to an upper bound (for example, we may have `num_rows = 256` and `num_cols = 512`). Then when we specify a tensor type as `[num_rows, num_cols]`, it corresponds to the tensor type $\mathcal{T}$ with $\mathcal{T}_r = 2$, $\mathcal{T}_l = \langle 0, 0 \rangle$, and $\mathcal{T}_u = \langle 256, 512 \rangle$.

We decompose computations over tensors so that they can be run in parallel. This is done by decomposing the original computation into smaller computations whose input and output types tile the input and output types to the original computation.

Definition 3.2 (Tiling). Given a tensor type $\mathcal{T}$, we say that the set of tensor types $T = \{\mathcal{T}_1, \mathcal{T}_2, \dots\}$ *tiles* $\mathcal{T}$ if (a) the indices of the tensor types in T cover the indices of $\mathcal{T}$; that is, $\mathcal{I}(\mathcal{T}) = \bigcup_i \mathcal{I}(\mathcal{T}_i)$ and (b) the indices of the tensor types in T are pairwise disjoint; that is, for all $i \neq j$, it is the case that $\mathcal{I}(\mathcal{T}_i) \cap \mathcal{I}(\mathcal{T}_j) = \emptyset$.

3.2 Abstract Operators and Join-Agg Specs

Einsummable programmers build computations using abstract operators over tensors. Abstract operators are operations such as matrix multiplication, flash attention, softmax, and so on. To a programmer, the actual implementation of the abstract operator (how its inputs are sharded, how it is parallelized) is invisible. In this subsection we formally define the notion of an abstract operator, as well as how they are implemented.

Definition 3.3 (Abstract operator). An *abstract operator* in Einsummable is a function over zero-bounded tensors. Given a list of zero or more zero-bounded input tensor types $\mathcal{T}^{(i_1)}, \mathcal{T}^{(i_2)}, \dots$, and a list of zero or more zero-bounded output tensor types $\mathcal{T}^{(o_1)}, \mathcal{T}^{(o_2)}, \dots$, an abstract operator is a function from the set $\mathcal{T}^{(i_1)} \times \mathcal{T}^{(i_2)} \times \dots$ to the set $\mathcal{T}^{(o_1)} \times \mathcal{T}^{(o_2)} \times \dots$.

Example 3.4. Consider matrix multiplication. If we have named indices `i`, `j`, and `k`, then matrix multiply is a function from `[i, j] × [j, k]` to `[i, k]`.

Every abstract operator can be decomposed and implemented as a *join* followed by an *aggregation*, in a way that is described using a *join-agg spec*. We first need three auxiliary notions.

Definition 3.5 (Projection; function specification; concrete operator). We say that a tensor type $\hat{\mathcal{T}}$ is a *projection* of a tensor type $\mathcal{T}$ if $\hat{\mathcal{T}}_r = \mathcal{T}_r$ and, componentwise, $\hat{\mathcal{T}}_l \leq \mathcal{T}_l$ and $\hat{\mathcal{T}}_u \leq \mathcal{T}_u$; note that then $\mathcal{I}(\hat{\mathcal{T}}) \subseteq \mathcal{I}(\mathcal{T})$. Given a tensor $\mathbf{U} \in \mathcal{T}$, the *projection of $\mathbf{U}$ along $\hat{\mathcal{T}}$* , denoted $\hat{\mathbf{U}}$, is the restriction of $\mathbf{U}$ to the domain $\mathcal{I}(\hat{\mathcal{T}})$. A *function specification* is an expression of the form $\mathcal{F} = \hat{\mathcal{T}}^{(i_1)} \times \hat{\mathcal{T}}^{(i_2)} \times \dots \rightarrow \hat{\mathcal{T}}^{(o_1)} \times \hat{\mathcal{T}}^{(o_2)} \times \dots$, that is, a list of input tensor types together with a list of output tensor types. Finally, a *concrete operator* f is a polymorphic function over tensors; we write $f[\mathcal{F}]$ for the instantiation of f at function specification $\mathcal{F}$, which is a function from $\hat{\mathcal{T}}^{(i_1)} \times \hat{\mathcal{T}}^{(i_2)} \times \dots$ to $\hat{\mathcal{T}}^{(o_1)} \times \hat{\mathcal{T}}^{(o_2)} \times \dots$.

Imagine that we decompose one or more of the tensors that are input into an abstract operator, so that each such input tensor is represented as a set of sub-tensors—that is, as a set of (sub_tensor_key, sub_tensor) pairs, where each key is the tensor type of the corresponding sub-tensor (a *tensor relation* [41]). The abstract operator is then implemented in three phases. If the abstract operator has n inputs, the first phase is a *join* where n -tuples of input tensors are formed from the decomposed inputs, and those n -tuples are sent to the abstract operator’s corresponding concrete operator f . Each invocation of f happens on a different processor (or AI accelerator). The second phase is an *aggregation* where some of the outputs of the various f invocations may need to be combined. And the final phase is the *recompose*, where the various partial results are stitched together to form the abstract operator’s final output. We now formally define a join-agg spec and its implementation.

Definition 3.6 (Join-agg spec). Consider an abstract operator of the form $\mathcal{T}^{(i_1)} \times \mathcal{T}^{(i_2)} \times \dots \rightarrow \mathcal{T}^{(o_1)} \times \mathcal{T}^{(o_2)} \times \dots$. A *join-agg spec* for this abstract operator is a set $\mathcal{J} = \{\mathcal{F}_1, \dots, \mathcal{F}_p\}$ of function specifications in which each input type $\hat{\mathcal{T}}^{(i_j)}$ is a projection of $\mathcal{T}^{(i_j)}$ and each output type $\hat{\mathcal{T}}^{(o_j)}$ is a projection of $\mathcal{T}^{(o_j)}$, and which satisfies the following *tiling condition*: for each output index j , the distinct j th output types appearing in $\mathcal{J}$ tile $\mathcal{T}^{(o_j)}$. When a join-agg spec is executed, each of its p function specifications is assigned to its own processor.

Definition 3.7 (Implementation). We say that a join-agg spec $\mathcal{J}$ implements an abstract operator if there exist a concrete operator f and a binary, associative, and commutative *aggregation operation* $\oplus$ over the reals (applied elementwise to tensors of a common type) such that, for every input to the abstract operator, the following three-step procedure recovers the correct output:

- (1) **Join.** For each $\mathcal{F} \in \mathcal{J}$: project each input tensor $\mathbf{U}^{(i_j)}$ along the corresponding $\hat{\mathcal{T}}^{(i_j)}$ from $\mathcal{F}$ to obtain $\hat{\mathbf{U}}^{(i_j)}$, then execute $f[\mathcal{F}]$ on $\hat{\mathbf{U}}^{(i_1)}, \hat{\mathbf{U}}^{(i_2)}, \dots$ to obtain $\hat{\mathbf{U}}^{(o_1)}, \hat{\mathbf{U}}^{(o_2)}, \dots$, where each $\hat{\mathbf{U}}^{(o_j)} \in \hat{\mathcal{T}}^{(o_j)}$.
- (2) **Aggregate.** For each output index j and each distinct type $\hat{\mathcal{T}}$ appearing as a j th output type in $\mathcal{J}$, combine using $\oplus$ the j th outputs of all function specifications whose j th output type is $\hat{\mathcal{T}}$, yielding a single tensor $\hat{\mathbf{V}} \in \hat{\mathcal{T}}$.
- (3) **Recompose.** For each output index j , the tiling condition ensures that there is a unique tensor $\mathbf{V}^{(o_j)} \in \mathcal{T}^{(o_j)}$ that agrees with each aggregated tensor from step (2) on that tensor’s index set; this $\mathbf{V}^{(o_j)}$ is the j th output.

Example 3.8 (Matrix multiply, decomposed). Let $i = k = 256$ and $j = 512$, and consider the matrix-multiply abstract operator of Example 3.4, from $[i, j] \times [j, k]$ to $[i, k]$. (This may be read as the first matrix multiply of the FFN of Section 2, with $i = \text{tokens}$, $j = \text{d_model}$, and $k = \text{d_ff}$.) A simple join-agg spec for this operator splits the shared index j into two halves. Let $\mathcal{J} = \{\mathcal{F}_1, \mathcal{F}_2\}$, where $\mathcal{F}_1 = \mathcal{A} \times \mathcal{B} \rightarrow \mathcal{C}$ and $\mathcal{F}_2 = \mathcal{A}' \times \mathcal{B}' \rightarrow \mathcal{C}$, where $\mathcal{A}$ and $\mathcal{A}'$ are the left and right halves (columns $[0, 256)$ and $[256, 512)$) of the first input type, $\mathcal{B}$ and $\mathcal{B}'$ are the top and bottom halves of the second, and $\mathcal{C}$ is the entire output type $[i, k]$. Each of these is a projection of the corresponding input or output type of the abstract operator, and since $\mathcal{C}$ is the only output type appearing in $\mathcal{J}$, with $\mathcal{I}(\mathcal{C}) = \mathcal{I}([i, k])$, the tiling condition holds trivially.

This join-agg spec implements matrix multiply. Take f to be the polymorphic block matrix multiply, which for a function specification $\hat{\mathcal{A}} \times \hat{\mathcal{B}} \rightarrow \hat{\mathcal{C}}$ multiplies the two projected blocks, contracting over the index range of $\hat{\mathcal{A}}$ ’s second dimension, and take $\oplus$ to be addition. In step (1) of Definition 3.7, the processor assigned $\mathcal{F}_1$ computes the partial products over the first half of j , and the processor assigned $\mathcal{F}_2$ over the second half. In step (2), the two results have the identical output type $\mathcal{C}$ and are summed elementwise. Step (3) is not necessary, as $\mathcal{C}$ is the only output block.

We can further decompose so as to require recomposition. Additionally split i into two halves. The resulting join-agg spec has four function specifications $\mathcal{F}_{m,n}$ for $m, n \in \{1, 2\}$, where $\mathcal{F}_{m,n}$ multiplies the block of the first input with rows in the m th half of i and columns in the n th half of j by the block of the second input with rows in the n th half of j , and has the single output type $\mathcal{C}_m$, with $(\mathcal{C}_1)_l = \langle 0, 0 \rangle$, $(\mathcal{C}_1)_u = \langle 128, 256 \rangle$, $(\mathcal{C}_2)_l = \langle 128, 0 \rangle$, and $(\mathcal{C}_2)_u = \langle 256, 256 \rangle$. Now two distinct output types appear in the spec. In step (2), the outputs of $\mathcal{F}_{m,1}$ and $\mathcal{F}_{m,2}$ are summed to produce the aggregated block of type $\mathcal{C}_m$, and in step (3) the two aggregated blocks, which tile the output index set, are stitched into the final output tensor.

3.3 Shardable Indices

One or more of the named indices in the specification for an abstract operator are *shardable*. Shardable indices are used by Einsummable to request join-agg specs from an abstract operator. Every shardable index will either have an associated list of positions that it can be sharded on, or else it will have an annotation any indicating that it can be sharded at any position. To request a join-agg spec from an abstract operator, Einsummable decomposes each shardable index into one or more intervals and provides the intervals to the abstract operator, which returns a corresponding join-agg spec. If $\text{intervals}[s]$ is the number of intervals associated with the s th shardable index, then the returned join-agg spec must contain exactly $\prod_s \text{intervals}[s]$ function specifications, one for each combination of intervals. Crucially, since the abstract operator is itself responsible for building the join-agg spec, there is a great deal of flexibility in how operators can be decomposed.

Example 3.9 (Matrix multiply). For the matrix multiply of Example 3.4, all three named indices i , j , and k are shardable with the annotation any, since a matrix multiply may be decomposed between any two rows or columns of its inputs. Now let $i = k = 256$ and $j = 512$. Suppose that during optimization, Einsummable decomposes i into the intervals $[0, 128)$ and $[128, 256)$, j into the intervals $[0, 256)$ and $[256, 512)$, and leaves k as the single interval $[0, 256)$. Since $\text{intervals}[i] = 2$, $\text{intervals}[j] = 2$, and $\text{intervals}[k] = 1$, the abstract operator must return a join-agg spec containing $2 \cdot 2 \cdot 1 = 4$ function specifications, and it returns the join-agg spec $\{\mathcal{F}_{m,n}\}$ of Example 3.8, where $\mathcal{F}_{m,n}$ corresponds to the m th interval of i paired with the n th interval of j . Note the differing roles of the indices. j indexes the contracted dimension, so specifications that share an i interval but differ in their j interval are combined by $\oplus = +$ during aggregation, whereas the decomposition of i passes through to an output tiling.

Example 3.10 (Flash attention). Consider flash-attention with grouped-query attention, with input types $[\text{heads_q}, \text{sequence},$

d_head] (the queries), [heads_kv, sequence, d_head] (the keys), and [heads_kv, sequence, d_head] (the values), and with output type [heads_q, sequence, d_head]. Let heads_q = 32 and heads_kv = 8, so that query heads $4m, \dots, 4m + 3$ are associated with the key-value head m . Of these named indices, only heads_kv and sequence are shardable. heads_q is *not* directly shardable, yet sharding heads_kv has the effect of sharding heads_q. If Einsummable decomposes heads_kv into its eight unit intervals, the abstract operator returns a join-agg spec with eight function specifications, where the m th specification projects the key and value inputs onto key-value head m and the query input and output onto the query-head interval $[4m, 4(m + 1))$. This logic is handled by the abstract operator when it produces the join-agg spec. This is why join-agg spec production is left to individual abstract operators. Decomposition logic can be operator specific and the Einsummable system does not need to directly reason about them.

The index sequence is an example of a shardable index without the any annotation. Suppose sequence = 4096 holds three independent sequences packed end to end, ending at positions 1024, 2560, and 4096. Attention never crosses a sequence boundary, but positions within a single sequence will attend to one another, so sequence may be sharded only at the ends of individual sequences.

4 LOGICAL OPTIMIZATION

4.1 Overview

Recall from Section 2 that compilation proceeds in two phases, which roughly correspond to logical and physical optimization in classical database systems. This section describes logical optimization, where Einsummable associates a join-agg spec with each abstract operator, chosen so as to minimize the communication associated with the computation. The result is a logical plan.

4.2 Choosing Join-Agg Specs

A computation is a directed, acyclic graph where vertices are abstract operators, and there is an edge (A, B) if abstract operator B consumes the output of A. For the time being we assume that each output is consumed only once, and that each operator produces only a single output. Logical optimization is powered by a dynamic programming algorithm operating over tilings. Let $\text{Opt}(\text{Op}, T)$ denote the lowest communication cost possible for computing the output of abstract operator Op , subject to the constraint that the tiling of the output is T . Assume we have a cost function $\text{Cost}(\mathcal{J}, T_1, T_2, \dots)$ that computes the communication cost required to implement join-agg spec $\mathcal{J}$ given that the first input has tiling T_1 , the second T_2 , and so on. This includes the cost of repartitioning each input from its tiling into the input projections required by $\mathcal{J}$, as well as the cost of the join and the aggregation; we describe the details subsequently. Further, for a join-agg spec $\mathcal{J}$, let $\text{out}(\mathcal{J})$ denote the tiling of the output induced by $\mathcal{J}$ (that is, the set of distinct output types appearing in $\mathcal{J}$ that tiles the output).

A key observation is that in $\text{Opt}(\text{Op}, T)$, the tiling T need only range over the output tilings that Op 's join-agg specs actually produce. It is never better to first repartition an output into some intermediate tiling and then repartition again at the consumer than it is to repartition directly. Consequently, nothing is lost by restricting the entries of the dynamic programming table for Op to the tilings

$\{\text{out}(\mathcal{J}) : \mathcal{J} \in \text{specs}(\text{Op})\}$. Note that this restriction does not determine the join-agg spec as several specs may produce the same output tiling. For example, in the matrix multiply of Example 3.9, specs that differ only in how the contracted index j is decomposed all induce the same tiling of the output.

Consider an abstract operator Op whose inputs are produced by abstract operators $\text{Op}_1, \dots, \text{Op}_n$, and let $\text{specs}(\text{Op})$ denote the set of join-agg specs over p processors that Op can produce. This set is obtained by requesting one join-agg spec from Op for each decomposition of its shardable indices into intervals whose counts multiply to p . The recurrence is

$$\text{Opt}(\text{Op}, T) = \min_{\substack{\mathcal{J} \in \text{specs}(\text{Op}) : \text{out}(\mathcal{J}) = T \\ T_1, \dots, T_n}} \left[\sum_{k=1}^n \text{Opt}(\text{Op}_k, T_k) + \text{Cost}(\mathcal{J}, T_1, \dots, T_n) \right],$$

where each T_k ranges over the candidate tilings of the corresponding producer Op_k . That is, to produce the output of Op tiled according to T , choose a join-agg spec $\mathcal{J}$ for Op that induces the tiling T , and choose a tiling T_k for each input; pay each producer Op_k to deliver its output tiled according to T_k , and pay to execute $\mathcal{J}$ on inputs that are so tiled (including the cost of repartitioning each input from T_k into the blocks that $\mathcal{J}$ requires). The base case is a tensor that is an input to the computation. Such a tensor may be materialized across the processors in any tiling, so $\text{Opt}(\text{Op}, T) = 0$ for every T when Op is an input.

The cost of the best logical plan is then $\min_T \text{Opt}(\text{Op}^*, T)$, where Op^* is the operator producing the computation's designated output tensor, and the plan itself (a choice of join-agg spec for every abstract operator) is recovered by tracing back the minimizing choices.

To fill the table for an operator, it is enough to compute, once per spec,

$$B(\mathcal{J}) = \min_{T_1, \dots, T_n} \left[\sum_{k=1}^n \text{Opt}(\text{Op}_k, T_k) + \text{Cost}(\mathcal{J}, T_1, \dots, T_n) \right],$$

after which $\text{Opt}(\text{Op}, T)$ is the minimum of $B(\mathcal{J})$ over the specs with $\text{out}(\mathcal{J}) = T$. Since the table for Op has one entry per distinct tiling, the number of entries is at most the number of specs. If each abstract operator has at most s join-agg specs and at most n inputs, so that each tensor has at most $t \leq s$ candidate tilings, the total running time is $O(\sum_{\text{Op}} s \cdot t^n)$, where the t^n factor comes from enumerating combinations of input tilings.

Algorithm 1 gives the complete procedure. It processes the operators bottom-up, computing $B(\mathcal{J})$ for each spec. For each output tiling, it records the cheapest join-agg spec that induces it, together with the input tilings that achieved the minimum. A final backwards pass walks the tree from the designated output.

4.3 Cost Model

Our implementation of $\text{Cost}(\mathcal{J}, T_1, \dots, T_n)$ does not take into account computation cost, as for every abstract operator available in the standard toolkit, the computational cost of implementing each join-agg spec for the operator is the same in terms of the number of floating point operations required. As Einsummable requests decompositions into near-equal intervals, the candidate join-agg

Algorithm 1 Logical optimization

Require: a computation tree whose leaves are input tensors and whose output is produced by Op^* , a processor count p

Ensure: a join-agg spec $\text{spec}[\text{Op}]$ for every abstract operator Op

```

1: for each input tensor  $X$  of the computation do
2:    $\text{cand}[X] \leftarrow \{\perp\}$ ;  $\text{Opt}[X, \perp] \leftarrow 0$ 
3: end for
4: for each operator  $\text{Op}$ , in bottom-up (topological) order do
5:   let  $\text{Op}_1, \dots, \text{Op}_n$  be the producers of the  $n$  inputs of  $\text{Op}$ 
6:    $\text{cand}[\text{Op}] \leftarrow \emptyset$ 
7:   for each  $\mathcal{J} \in \text{specs}(\text{Op})$  do
8:     choose  $(T_1^*, \dots, T_n^*) \in \text{cand}[\text{Op}_1] \times \dots \times \text{cand}[\text{Op}_n]$  to
        $\min B \leftarrow \sum_{k=1}^n \text{Opt}[\text{Op}_k, T_k^*] + \text{Cost}(\mathcal{J}, T_1, \dots, T_n)$ 
9:      $T \leftarrow \text{out}(\mathcal{J})$ 
10:    if  $T \notin \text{cand}[\text{Op}]$  or  $B < \text{Opt}[\text{Op}, T]$  then
11:       $\text{cand}[\text{Op}] \leftarrow \text{cand}[\text{Op}] \cup \{T\}$ 
12:       $\text{Opt}[\text{Op}, T] \leftarrow B$ 
13:       $\text{choice}[\text{Op}, T] \leftarrow (\mathcal{J}, T_1^*, \dots, T_n^*)$ 
14:    end if
15:  end for
16: end for
17:  $T^* \leftarrow$  the  $T \in \text{cand}[\text{Op}^*]$  minimizing  $\text{Opt}[\text{Op}^*, T]$ 
18:  $\text{ASSIGN}(\text{Op}^*, T^*)$ 

19: procedure  $\text{ASSIGN}(\text{Op}, T)$ 
20:    $(\mathcal{J}, T_1, \dots, T_n) \leftarrow \text{choice}[\text{Op}, T]$ 
21:    $\text{spec}[\text{Op}] \leftarrow \mathcal{J}$ 
22:   for  $k = 1, \dots, n$  do
23:     if the  $k$ th input of  $\text{Op}$  is produced  $\text{Op}_k$  then
24:        $\text{ASSIGN}(\text{Op}_k, T_k)$ 
25:     end if
26:   end for
27: end procedure

```

specs are load-balanced and so the computation time is effectively the same across the candidate specs.

However, different join-agg specs can have very different communication costs. Thus Cost computes a proxy for communication, just as a logical query optimizer in a relational database system costs candidate plans using a proxy such as the number of intermediate tuples produced, deferring precise, hardware-aware costing to physical optimization. Our Cost function estimates the number of floating point numbers that may need to be transferred to run the join-agg spec $\mathcal{J}$, ignoring which processor each block of data actually resides on; data placement is considered during physical optimization. Assuming that each abstract operator produces a single output, the estimate includes three separate sub-costs:

- (1) **Repertition.** Consider the k th input tensor to the operator in question, with type $\mathcal{T}^{(i_k)}$, and let $\text{in}_k(\mathcal{J})$ denote the set of distinct k th input types appearing in the function specifications of $\mathcal{J}$. If $\text{in}_k(\mathcal{J}) \neq T_k$, then add $|\mathcal{I}(\mathcal{T}^{(i_k)})|$ to the overall cost. Intuitively: if we have to repartition the tensor, we may have to move every floating point number in the tensor around the network; there are $|\mathcal{I}(\mathcal{T}^{(i_k)})|$ such

numbers. Note that if the k th input is an input to the computation the tensor is materialized directly into the blocks required by $\mathcal{J}$, and this sub-cost is zero.

- (2) **Join.** Consider each function specification in $\mathcal{J}$ of the form $\hat{\mathcal{T}}^{(i_1)} \times \hat{\mathcal{T}}^{(i_2)} \times \dots \rightarrow \hat{\mathcal{T}}^{(o)}$. Simply add $\sum_k |\mathcal{I}(\hat{\mathcal{T}}^{(i_k)})|$ to the overall cost. Intuitively: we have to form every n -tuple that is the input to some invocation of the concrete operator f , and this counts the total number of floating point numbers in those n -tuples. This penalizes join-agg specs that require large inputs to f .
- (3) **Aggregation.** Consider each distinct type $\hat{\mathcal{T}}^{(o)}$ appearing as an output type in $\mathcal{J}$. Count the number of times this distinct type appears in $\mathcal{J}$; let this be m . Add $(m-1)|\mathcal{I}(\hat{\mathcal{T}}^{(o)})|$ to the overall cost. Intuitively: these m copies need to be aggregated down to one copy, which (naively) requires moving $(m-1)$ copies to the location of the last copy, and this counts the total number of floating point numbers moved.

4.4 Detailed Example: 3D Matrix Multiply

We now use this machinery end to end, and show that the optimizer recovers the classical “3D” matrix-multiplication algorithm [1, 9], which is communication-optimal for square matrices [16].

Consider a computation consisting of a single matrix multiply (Example 3.4) with $i = j = k = N$, whose two input matrices are inputs to the computation, to be run on p processors; for simplicity, assume $p = q^3$ for an integer q , and that the interval counts under consideration divide N . As in Example 3.9, every candidate join-agg spec for this operator decomposes i, j , and k into a, b , and c equal intervals with $abc = p$, and contains one function specification per cell of the resulting $a \times b \times c$ grid. The specification for a given cell multiplies an $(N/a) \times (N/b)$ block of the first input by an $(N/b) \times (N/c)$ block of the second producing a partial result for an $(N/a) \times (N/c)$ block of the output.

We evaluate Cost on such a spec. Sub-cost (1) is zero as both inputs are inputs to the computation. For sub-cost (2) each of the abc function specifications has input blocks totaling $\frac{N^2}{ab} + \frac{N^2}{bc}$ numbers, for a total of $abc \left(\frac{N^2}{ab} + \frac{N^2}{bc} \right) = (a+c)N^2$. Intuitively, each element of the first input is replicated c times, and each element of the second input a times. For sub-cost (3), the distinct output types are the ac blocks of an $a \times c$ tiling of the output, and each appears $m = b$ times, contributing $(b-1)N^2$. Hence

$$\text{Cost} = (a+c)N^2 + (b-1)N^2 = (a+b+c-1)N^2.$$

By the arithmetic/geometric mean inequality, $a+b+c \geq 3\sqrt[3]{abc} = 3q$, with equality exactly when $a = b = c = q$. The optimizer therefore selects the $q \times q \times q$ grid, at a cost of $(3q-1)N^2 = (3p^{1/3}-1)N^2$ (each processor multiplies an $(N/q) \times (N/q)$ block of the first input by an $(N/q) \times (N/q)$ block of the second, and the q partial results for each output block are summed). This is precisely the 3D matrix-multiplication algorithm [1].

4.5 Edge Cases

The discussion so far assumes that each abstract operator has one output, consumed by one operator. Three extensions cover the cases that arise in practice. If an operator has several outputs that all flow to the same consumer, Opt is indexed by a *tuple* of tilings,

one per output, and the consumer selects a single table entry of that producer rather than choosing the tilings independently; the graph remains a tree and optimal substructure is preserved. If an output has several consumers, the computation is no longer a tree, and an exact algorithm would have to track the joint demands on every shared operator, a state space that grows exponentially with the amount of sharing. Instead, the graph is cut at every edge out of a multiply-consumed tensor, and the resulting trees are optimized with Algorithm 1 one at a time in decreasing order of size, each subject to the join-agg specs already fixed (a tensor produced in an already-optimized tree enters the recurrence like a computation input, except that Cost charges any repartitioning). Finally, because the per-operator work is $O(s \cdot t^n)$ in the number of inputs n , our implementation applies the same tactic to linear paths of operators: each operator on a path draws at most one input from within the path, so the inner minimization in $B(\mathcal{J})$ ranges over a single producer’s tilings and the work drops to $O(s \cdot t)$. In both cases the plan is optimal within each piece, though not necessarily globally.

5 PHYSICAL OPTIMIZATION

5.1 Overview

Einsummable’s physical optimizer accepts the output of the logical optimizer (a DAG of abstract operators, each labeled with a selected join-agg spec) and produces an executable artifact. In the current version of Einsummable, this artifact is an NVIDIA CUDA graph, in which each vertex is a GPU kernel or a network operation, which can be executed directly on an NVIDIA server.

Physical optimization proceeds through the graph of abstract operators in topologically sorted order, so that when an operator op is reached, every operator supplying one of its inputs has already been compiled. Each producer is compiled *through its join* so that when op is reached, the unaggregated partial blocks of each of op ’s inputs are pinned to specific GPUs. It is op ’s job to aggregate those partial blocks as it brings them where they are needed.

To do so, the compiler generates the *preamble* of a program in a domain-specific language called ExProg (for *exchange program*). This includes an IN declaration recording the location and size of every partial block, and an OUT declaration, derived from op ’s join-agg spec, requiring each aggregated input block at the sites that will run op ’s concrete operator. The sites in the OUT declaration are expressed as partitions declared by size, so that the placement of op ’s p concrete-operator instances is itself chosen during synthesis. Exchange programs are a topology-aware generalization of Volcano’s exchange operator [12]. As with the exchange operator, an exchange program is where repartitioning, replication, and inter-processor parallelism are visible. Unlike the exchange operator, an exchange program also performs aggregation, and it is written against an explicit model of the hardware topology.

Given the preamble, the body of the exchange program is synthesized so as to minimize the execution time predicted by a hardware simulator. The compiler uses the synthesized program and its simulated execution trace (fixing the dependency structure among transfers, aggregations, and compute kernels) to create the portion of the CUDA graph that links op to its inputs and invokes each of the p instances of the concrete operator f implementing op .

5.2 The ExProg Language

State. An exchange program manipulates named *atoms* which are tensors, each with a size (in GB). The state of the machine at any instant is a set of pairs (*aggregate*, *site*), where an aggregate is a $\oplus$ -combination of atoms, written $A+B+C$, and a site is a GPU. Because $\oplus$ is associative and commutative, $A+B$ and $B+A$ denote the same aggregate; the symbol $+$ denotes the aggregation operation $\oplus$ of the operator at hand, which need not be addition. A program declares an initial state and a required final state, and its body must transform the former into the latter.

- **GPUS:** lists the valid sites, and **PARTITIONS:** optionally names groups of sites; the groups must be disjoint and cover the declared GPUs. A partition may also be declared by size alone, as in $G1 \{size\ 4\}$, in which case the compiler chooses its membership, and the final state must hold for some choice of membership.
- **IN:** places each atom at a site, optionally with a size, e.g., $IN: A @ 0 \ (size: 1.0\ GB)$ (sizes default to 1 GB).
- **OUT:** lists the aggregates that must exist, and at which sites or partitions, e.g., $OUT: A+B @ 0, 1$.

Actions. The body of a program is a set of *actions*, each naming an aggregate, the site at which it is formed, and an optional destination list. $A+B @ 0 \rightarrow 1$ aggregates A and B at site 0 and sends the result to site 1. $A @ 0 \rightarrow 1, 2, 3$ multicasts A to three sites. $A+B @ 0$ aggregates locally with no network traffic. $A+B @ G1$ aggregates at every site of partition $G1$ simultaneously. When convenient, we write several actions on one line, separated by semicolons. There are three key rules. First, *a program is a set, not a sequence*. An action becomes eligible the moment every atom of its aggregate is present at its source, and all eligible actions proceed concurrently. Intermediate states emerge from dataflow, not program order. Second, every action with an arrow is a two-stage pipeline: a local aggregation (reads and writes at HBM bandwidth), followed by a transfer across the link. Third, aggregating at the source materializes the aggregate there, satisfying the final-state requirement at that site, and other actions may reuse it.

Cost. A hardware simulator assigns each program a completion time. DGX-A100 and DGX-H100 machines are modeled as fully non-blocking all-to-all NVSwitch fabrics in which the only contention is at endpoints. A multicast to k destinations divides the source’s egress bandwidth by k . The DGX-V100 is far more restrictive as its eight GPUs form a hybrid cube-mesh with 50 GB/s double NVLinks on some pairs, 25 GB/s single links on others, and no direct link on the rest so transfers between unlinked pairs fall back to a PCIe bridge with 15.75 GB/s of total capacity shared by all such transfers.

Example 5.1 (A peer-to-peer swap). Two GPUs each hold one atom, and both must end up with the aggregate:

```
IN:  A @ 0; B @ 1
OUT: A+B @ 0, 1
```

```
A @ 0 -> 1; B @ 1 -> 0; A+B @ 0, 1
```

Both transfers are eligible immediately and proceed concurrently and when complete, the aggregations run. On a DGX-H100 with 1

GB atoms, the simulator reports completion in roughly 3.1 milliseconds with 2.2 ms for the (overlapping) transfers, and the remainder for the local aggregations.

Example 5.2 (Recursive doubling). Four GPUs each hold one atom, and all four must obtain the total aggregate:

IN: A @ 0; B @ 1; C @ 2; D @ 3
OUT: A+B+C+D @ 0, 1, 2, 3

A @ 0 -> 1; B @ 1 -> 0
C @ 2 -> 3; D @ 3 -> 2
A+B @ 0 -> 2; A+B @ 1 -> 3
C+D @ 2 -> 0; C+D @ 3 -> 1
A+B+C+D @ 0, 1, 2, 3

The first four actions are eligible at time zero. The second group becomes eligible as its operands arrive. Partial aggregates cross the network in the second round, halving the traffic relative to gathering all four atoms everywhere. On a DGX-H100 with 1 GB atoms, the simulator reports completion in 6.2 milliseconds.

Example 5.3 (Topology-aware placement on a DGX-V100). The following specification leaves placement to the compiler: the two partitions are declared by size alone, so the final state must hold only for some choice of two four-GPU groups.

GPUS: 0, 1, 2, 3, 4, 5, 6, 7
PARTITIONS: G1 {size 4} G2 {size 4}
IN: A @ 0; B @ 1; C @ 2; D @ 3;
E @ 4; F @ 5; G @ 6; H @ 7
OUT: A+B+E+F @ G1; C+D+G+H @ G2

On a DGX-V100, the compiler chooses $G1 = \{0, 1, 4, 5\}$ and $G2 = \{2, 3, 6, 7\}$, aligning each group with the machine’s 50 GB/s double-links ($0 \leftrightarrow 4$, $1 \leftrightarrow 5$, $2 \leftrightarrow 6$, $3 \leftrightarrow 7$), and emits:

A @ 0 -> 1; B @ 1 -> 0; C @ 2 -> 3
D @ 3 -> 2; E @ 4 -> 5; F @ 5 -> 4
G @ 6 -> 7; H @ 7 -> 6
A+B @ 0 -> 4; A+B @ 1 -> 5; C+D @ 2 -> 6
C+D @ 3 -> 7; E+F @ 4 -> 0; E+F @ 5 -> 1
G+H @ 6 -> 2; G+H @ 7 -> 3
A+B+E+F @ G1; C+D+G+H @ G2

The first round of swaps uses the 25 GB/s single links within each CPU socket. The second round crosses the socket boundary on the 50 GB/s pillars and no transfer uses the 15.75 GB/s PCIe bridge. The simulator reports completion in roughly 67 milliseconds.

5.3 From Join-Agg Specs to Exchange Programs

As Einsumable’s physical optimizer proceeds through the directed, acyclic graph of abstract operators, for a given abstract operator op, the optimizer must produce the preamble for the exchange program used to physically implement op. To do this, the compiler must determine the atoms that the program will operate over. These atoms represent the finest pieces of data that the exchange program can possibly handle so that the exchange program can do its work by aggregating and routing whole atoms. Splitting and concatenation happen only just before or just after the compiled program is executed, in kernel calls generated by the compiler.

Consider the k th input of op, with tensor type $\mathcal{T}^{(i_k)}$ produced by an already-compiled operator whose join-agg spec is $\mathcal{J}_k$. Two

families of blocks must be considered. First, the distinct output types of $\mathcal{J}_k$ tile $\mathcal{T}^{(i_k)}$, and one copy of each such block per function specification of $\mathcal{J}_k$ that lists it exists somewhere on a GPU. Second, op’s join-agg spec $\mathcal{J}$ requires the blocks $\text{in}_k(\mathcal{J})$. These must be delivered, fully aggregated, to the GPU of every function specification of $\mathcal{J}$ consuming it. To ensure that no blocks need to be split during the exchange, the compiler forms the *overlay* of the two. In each dimension, it collects the lower and upper bounds, in that dimension, of every block in both families. The resulting grid of tensor types tiles $\mathcal{T}^{(i_k)}$ and its cells are the atoms for input k . Every physically existing block and every required block is a disjoint union of atoms.

The compiler brackets the exchange program with local kernel calls. Before the program runs decompose kernels slice each resident copy of a producer output block into copies of the atoms it contains. After the program runs, recompose kernels concatenate the aggregated atoms delivered to each GPU into the contiguous input blocks expected by the concrete operator f .

Given this, the IN declaration lists one ExProg atom per (atom, copy) pair at the GPU holding that copy, with size equal to the cardinality of the atom’s index set times the width of the datatype. The PARTITIONS declaration creates one partition of size one per function specification of op. The OUT declaration requires, for each atom, the $\oplus$ -aggregate of its m copies at the partition of every function specification of op whose input blocks contain that atom.

Example 5.4 (Generating a preamble). Suppose the producer is the matrix multiply of Example 3.8 compiled with its four-specification join-agg spec $\{\mathcal{F}_{m,n}\}$, and that the compiler assigned $\mathcal{F}_{1,1}, \mathcal{F}_{1,2}, \mathcal{F}_{2,1}, \mathcal{F}_{2,2}$ to GPUs 0, 1, 2, 3. Then GPUs 0 and 1 each hold an unaggregated copy of the output block C_1 (rows [0, 128)) and GPUs 2 and 3 each hold a copy of C_2 (rows [128, 256)). The consumer op is a row-wise softmax whose join-agg spec decomposes i into four intervals of 64 rows, so that $\text{in}_1(\mathcal{J})$ consists of the four blocks with rows [0, 64), [64, 128), [128, 192), [192, 256), each consumed by exactly one function specification. The overlay has row split points {0, 64, 128, 192, 256}, giving four atoms. Each producer block is the union of two atoms (which we call X_j and Y_j for the two copies of atom j). The generated preamble is:

GPUS: 0, 1, 2, 3
PARTITIONS: S1 {size 1} S2 {size 1}
S3 {size 1} S4 {size 1}
IN: X1 @ 0; X2 @ 0; Y1 @ 1; Y2 @ 1;
X3 @ 2; X4 @ 2; Y3 @ 3; Y4 @ 3
OUT: X1+Y1 @ S1; X2+Y2 @ S2;
X3+Y3 @ S3; X4+Y4 @ S4

Synthesis chooses $S1, \dots, S4$ and the program body.

5.4 Synthesizing Program Bodies

Given a preamble we may synthesize the body in a number of ways. For example, we may start with a trivial, valid program, and repeatedly rewrite to a new valid program, using the simulator to cost each candidate. Regardless of the algorithm that chooses the order of transformations applied, they must be sound (they never produce an invalid program) and complete (every valid program is reachable, so in particular a cost-optimal one is). We now define a set of sound and complete transformations.

Definition 5.5 (Closure; validity). A *state* is a set of pairs (X, s) , where X is an aggregate and s is a site. The initial state is $S_{\text{IN}} = \{(\{A\}, s) : A @ s \in \text{IN}\}$. An action $X @ s \rightarrow D$ is *enabled* by a state S if S contains aggregates $Y_1, \dots, Y_m$ at site s that partition X (that is, $X = Y_1 \cup \dots \cup Y_m$ with the Y_i pairwise disjoint). Firing it adds (X, s) and (X, d) for each $d \in D$. The *closure* $S^*(P)$ of a program P is the least state that contains S_{IN} and is closed under firing the enabled actions of P . P is *well-formed* if every action of P is enabled by $S^*(P)$, and *valid* if it is well-formed and $(X, s) \in S^*(P)$ for every requirement $X @ s$ of OUT.

For partitions declared by size alone, a program is valid if some choice of membership makes it valid; below we treat the membership as fixed. A *transformation* is a rule that maps a program to a program, and it is *sound* if it maps every valid program to a valid program for the same IN and OUT. Two primitive transformations suffice for completeness. $\text{INSERT}(a)$ maps P to $P \cup \{a\}$ and is permitted whenever the result is well-formed. $\text{DELETE}(a)$ maps P to $P \setminus \{a\}$ and is permitted whenever the result is valid. Both are sound. Deletion is always sound due to the guard, and insertion because of monotonicity (adding an action can only grow the closure).

THEOREM 5.6 (COMPLETENESS). *Let P and Q be valid programs for the same IN and OUT. Then P can be transformed into Q by $|Q \setminus P|$ insertions followed by $|P \setminus Q|$ deletions, with every intermediate program valid.*

PROOF. Insert the actions of $Q \setminus P$ in a firing order of Q ; enabledness is monotone in the program, so each insertion is permitted, and this reaches $P \cup Q$. Then delete the actions of $P \setminus Q$ latest-firing first: every intermediate contains Q , so OUT stays satisfied, and every remaining action is enabled by actions that fire strictly earlier and are still present. $\square$

Since a valid program always exists whenever OUT is satisfiable at all (gather the atoms of each required aggregate at each of its target sites and aggregate locally), the theorem implies that all valid programs are reachable from any valid starting program.

INSERT and DELETE are used merely to argue completeness and no reasonable search strategy would use them. We give a more reasonable set of rules below. Note that these are not unconditional rewrites. Deleting an action also deletes the materializations it produced, and those materializations may be needed elsewhere, by an OUT requirement, or to enable another action.

- *Multicast fission and fusion:* $X @ s \rightarrow D_1 \cup D_2 \leftrightarrow$ the pair $X @ s \rightarrow D_1$ and $X @ s \rightarrow D_2$, trading egress contention against action count. Both directions leave the closure unchanged, so both are always permitted.
- *Relay introduction and elimination:* $X @ s \rightarrow d \leftrightarrow X @ s \rightarrow r \rightarrow d$. This is a “bounce” move that routes around slow or contended links. Introduction only adds a materialization at r and is always permitted; elimination is permitted only when X at r is not otherwise required.
- *Aggregation pushdown and pullup:* the pair $X @ r \rightarrow s, Y @ r \rightarrow s \leftrightarrow X+Y @ r \rightarrow s$. This is the network analogue of eager versus lazy partial aggregation [40]. Both directions are guarded: pushdown deletes the separate materializations of X and Y at s , while pullup deletes that of $X+Y$ (a local aggregation $X+Y @ s$ may be inserted to restore it).

- *Source substitution:* redirect a transfer of X to draw from another site at which X is materialized. Guarded: the redirected transfer no longer materializes X at the original source.

In our implementation, Definition 5.5 ensures the validity of every candidate exchange program that the search procedure proposes, and Theorem 5.6 guarantees the space of valid programs being searched contains a cost-optimal program.

6 EXPERIMENTAL EVALUATION

Our evaluation asks three questions. First, can our automatic system meet the performance of mature systems expertly tuned? Second, how effective is Einsummable’s optimization: how important is cost-based selection of join-agg specs (Section 4), and how much does the synthesis of exchange programs matter (Section 5)? Third, is the communication-cost proxy of Section 4.3 reasonable?

Einsummable is implemented in C++ using CUDA. Concrete operators are high-performance kernels: cuBLAS matrix multiplies, FlashAttention-style attention kernels, and Triton-generated kernels for elementwise operations, aggregation, and the decomposition/recomposition of atoms (Section 5.3). Every plan is compiled to a CUDA graph and executed directly. Exchange-program synthesis uses an off-the-shelf commercial LLM to propose candidate programs. Each candidate is checked for validity (Definition 5.5), costed by the hardware simulator, and the cheapest is kept.

Experiments are run on two servers. Most experiments use an NVIDIA DGX A100 with eight 40 GB A100 GPUs connected by a fully non-blocking NVSwitch. Experiment 4 additionally uses an NVIDIA DGX V100 with eight 32 GB V100 GPUs, whose hybrid cube-mesh topology (Section 5.2) is challenging. All computations run in FP16 and are executed with two warm-up rounds followed by ten timed rounds. Figures report mean runtimes in milliseconds with 95% confidence intervals (which are vanishingly small).

Two workload families are used throughout. The first is a LLaMA-scale transformer block [13]: model dimension 4,096, thirty-two query heads and eight key/value heads (grouped-query attention, as in Example 3.10) of head dimension 128, and FFN dimension 14,336, with attention computed by fused FlashAttention-style kernels. The block is evaluated on five token workloads: single sequences of 1K, 8K, and 128K tokens, and batches of eight and 128 sequences of 1K tokens each. The second family is five fixed matrix chains, labeled A through E, designed so that each multiply averages roughly 8192^3 scalar multiplies while stressing different plan structures: chains A and B have *bushy* optimal parenthesizations (three and one independent sub-products, respectively, verified by the classical matrix-chain dynamic program); chain C is a three-matrix chain whose optimal order is right-associated, the opposite of what left-deep data parallelism produces; and chains D and E have large end matrices multiplied across thin interior dimensions, so that each plan is forced into a single large terminal aggregation. The parenthesization is fixed and identical for every system. Writing M_i for the i th matrix of a chain, the parenthesizations are: Chain A, $(M_1 M_2) (((M_3 M_4) ((M_5 M_6) (M_7 M_8))) M_9)$, nine matrices with dimensions alternating 16,384 and 6,144 between ends of 10,240; Chain B, $(M_1 M_2) ((M_3 M_4) M_5)$, five matrices with dimensions alternating 20,480 and 5,120 between ends of 9,216; Chain C, $M_1 (M_2 M_3)$, with dimensions 10,240×12,288×32,768×2,048; Chain D,

Table 1: Pearson correlation between predicted communication cost and measured runtime.

Transformer workload	r	Matrix chain	r
1 seq $\times$ 1K tokens	0.72	Chain A	0.73
1 seq $\times$ 8K tokens	0.92	Chain B	0.81
1 seq $\times$ 128K tokens	0.80	Chain C	0.87
8 seq $\times$ 1K tokens	0.85	Chain D	0.47
128 seq $\times$ 1K tokens	0.75	Chain E	0.92

$M_1 ((M_2 (M_3 (M_4 (M_5 (M_6 (M_7 M_8)))))) M_9)$, nine matrices with ends of 24,576, necks of 1,536, and a 20,480-wide interior; and Chain E, $M_1 ((M_2 (M_3 (\dots (M_{11} M_{12}) \dots)) M_{13}))$, thirteen matrices with ends of 49,152, necks of 1,024, and a 20,480-wide interior.

6.1 Experiments Run

Experiment 1: Comparing Einsummable with mature, expertly-tuned systems on transformer workloads. We compare against JAX [4] (XLA-compiled, with GSPMD sharding annotations [39] and cuDNN fused attention), PyTorch [27] (NCCL communication, flash scaled-dot-product attention kernels, weights replicated and activations sharded by batch or by sequence, whichever is better for the workload), and vLLM’s FlashAttention-2 kernels [24] driven directly. Each baseline was tuned by hand; Einsummable is fully automatic. All four systems run all five token workloads on one, four, and eight GPUs, except that JAX was unable to complete the 128K single-sequence workload. Results for the four larger workloads are given in Figure 1. On the 1K single-sequence workload, omitted from the figure for space, every system finishes in 0.8–2.2 ms at eight GPUs (JAX 0.83, Einsummable 1.21, PyTorch 1.72, vLLM 2.19), and it is included in the geometric means reported below.

Experiment 2: Comparing Einsummable with mature systems on matrix-chain arithmetic. On chains A through E we compare against JAX (JIT-compiled, row-sharded) and PyTorch (cuBLAS multiplies with NCCL row sharding), again on one, four, and eight GPUs. Results are given in Figure 2.

Experiment 3: Testing the value of coordinated, cost-based logical optimization. We run the transformer block and the matrix chains on eight GPUs under three plan selectors, all with physical optimization on: (a) the full logical optimizer of Section 4 (labeled “EinDecomp” in the figures, after the decomposition algorithm [3] on which it is based), (b) a *greedy* selector, which is the same dynamic program with the repartition sub-cost removed (since repartitioning is the only sub-cost that depends on the input tilings, this decouples the recurrence and each operator independently chooses its cheapest join-agg spec), and (c) *random* selection, which chooses uniformly among valid join-agg specs, with twenty independent draws per workload. Results are given in Figures 3 and 4.

Experiment 4: Testing the value of physical optimization on a restrictive topology. We compare optimized exchange programs against naive ones (each aggregated input block is simply gathered and aggregated at its consuming sites, with no topology-aware routing, relaying, or placement) for the transformer block and the matrix chains on the DGX V100, which has a challenging network topology. Results are given in Figures 5 and 6. For calibration, we also ran the same comparison on the NVSwitch-based DGX A100.

Experiment 5: Validating the communication-cost proxy. The twenty random plans per workload from Experiment 3 give, for each of the ten (workload, chain) setups, twenty (predicted cost, measured runtime) pairs. Because FLOPs are invariant across decompositions, runtime should be a linear-plus-noise function of communication, so we report the Pearson correlation (Table 1).

6.2 Discussion

Over all experiments, fully automatic Einsummable met or exceeded the performance of every system tested. On the transformer block at eight GPUs, Einsummable’s geometric-mean runtime over the five workloads is 8.97 ms, versus 13.65 ms for hand-tuned PyTorch and 14.87 ms for vLLM.

In **Experiment 1**, the gap is biggest on the long-sequence workloads. On the single 128K-token sequence, Einsummable requires 143 ms where PyTorch requires 506 ms and vLLM 498 ms (roughly 3.5 $\times$ longer). The plan Einsummable selects for this workload is instructive. A single sequence offers exactly one eight-way decomposition of the attention operator. Shard heads_kv, so that each GPU computes attention for one key/value head and its four query heads over the entire sequence. The token activations ($131,072 \times 4,096$, about 1 GB in FP16) dwarf every weight matrix, so the optimizer shards the normalization, elementwise, projection, and FFN operators by tokens with replicated weights, and connects the token-sharded and head-sharded regions by repartitioning the activations at the boundaries of the attention operator rather than sharding the contraction of the output projection. This is the hand-designed DeepSpeed-Ulysses scheme [17] (sequence parallelism outside attention, head parallelism inside it, joined by all-to-all exchanges) which Einsummable derives automatically. With a batch of one, the PyTorch implementation can only shard activations by sequence, so every GPU must gather the full keys and values and absorb the causal-mask load imbalance across sequence shards.

In **Experiment 2**, the three systems are effectively tied. At eight GPUs, every per-chain difference is within about 6%, and all three scale by roughly 6 $\times$ from one to eight GPUs. We believe that Einsummable’s parity is actually a strong result. With no manual decomposition of any kind, it handles classical workloads as well as systems with many person-years of development effort do.

Experiment 3 shows that random plan selection is catastrophic on the transformer block. The median random plan has a geometric-mean runtime of 22.8 ms against the optimizer’s 8.97 ms (2.5 $\times$ worse), and even the best of twenty draws is 17.6 ms. On the matrix chains the penalty for a poor logical plan is smaller. The median random plan is 34% worse in geometric mean, while the best of twenty draws essentially matches the optimizer (within 3%, occasionally beating it on an individual chain). The greedy selector is more interesting. In geometric mean it costs only 2.3% on the transformer block (and on a few individual workloads it is slightly faster, within the noise) but 7% on the chains. This may suggest that exchange programs are able to effectively mask the added cost associated with operator-to-operator repartitions.

Experiment 4 confirms that physical optimization can be crucial. On the NVSwitch-based A100, where every pair of GPUs enjoys a dedicated, non-blocking path, optimized and naive exchange programs are statistically indistinguishable (geometric-mean ratio

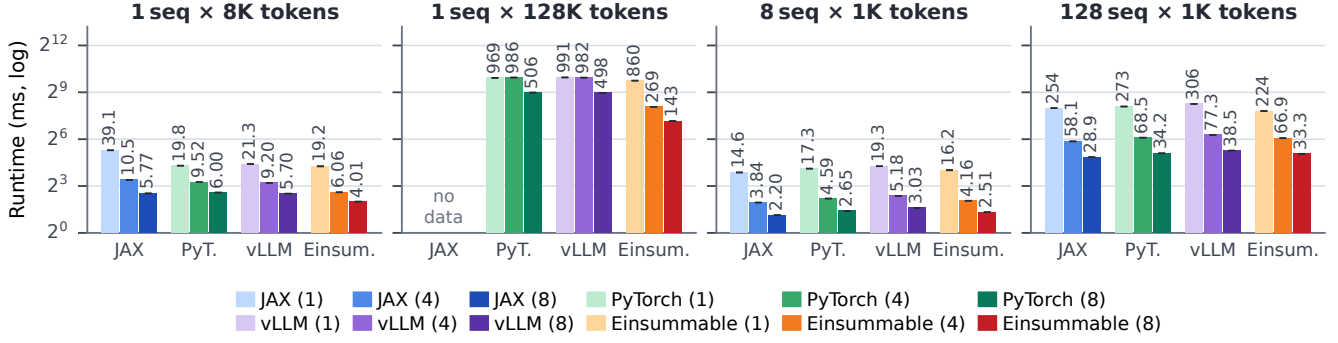


Figure 1: Transformer-block mean runtimes (95% CIs) for Einsummable vs. JAX, PyTorch, and vLLM on one, four, and eight A100 GPUs (parenthesized numbers in the legend), four LLM workloads.

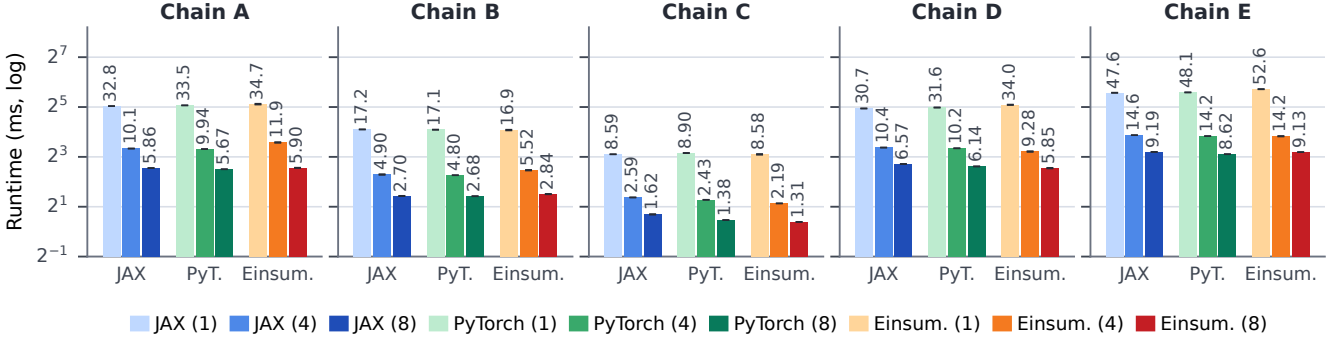


Figure 2: Matrix-chain mean runtimes (95% CIs) for Einsummable vs. JAX and PyTorch on one, four, and eight A100 GPUs (parenthesized numbers in the legend), over chains A to E.

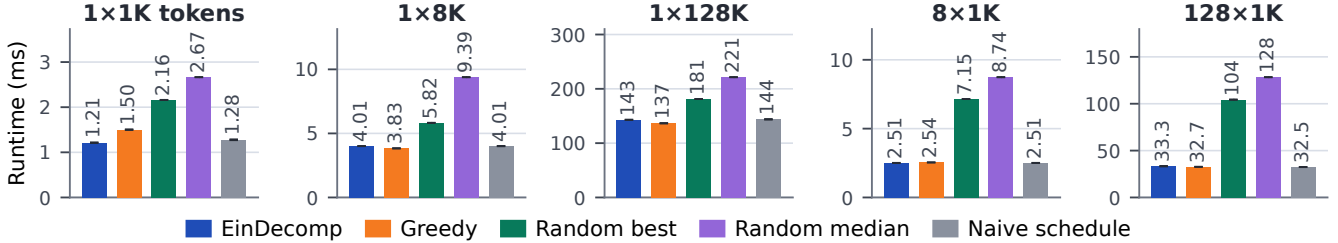


Figure 3: Logical-optimization ablation on the transformer block (eight A100 GPUs): the full optimizer vs. greedy, random, and naive-schedule plan selection.

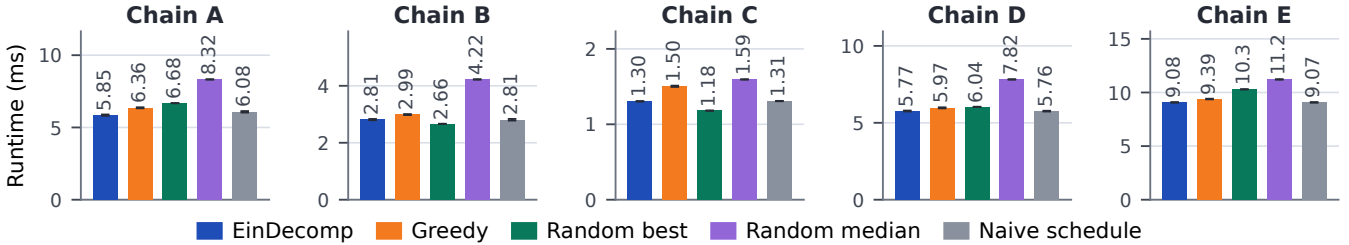


Figure 4: Logical-optimization ablation on chains A-E (eight A100 GPUs).

1.006 on the transformer block). With no topology to exploit and the logical optimizer having already removed unnecessary repartitions, a naive solution does well and there is little improvement possible beyond the naive solution. However, on the V100’s hybrid cube-mesh, optimized programs are up to 11% faster on the transformer

block (geometric mean 5.6%), and 1.41× and 1.32× faster on chains A and B, whose bushy plans generate cross-socket traffic that the naive programs route through the shared PCIe bridge. On modern hierarchical networks (multi-node NVLink domains, for example) we expect a similar advantage.

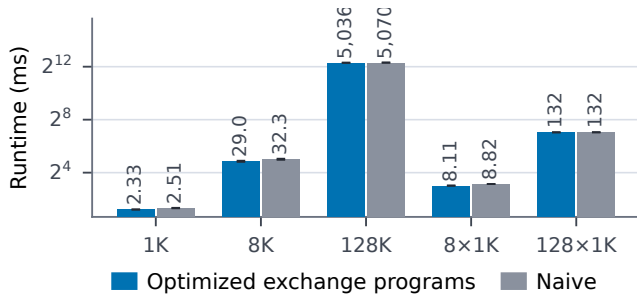


Figure 5: Optimized vs. naive exchange programs for the transformer block on the topology-restricted DGX V100.

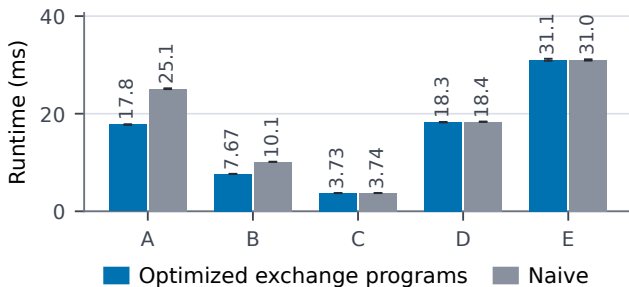


Figure 6: Optimized vs. naive exchange programs, DGX V100.

In **Experiment 5**, across the ten setups, the Pearson correlation between predicted communication cost and measured runtime ranges from 0.47 to 0.92, with eight of ten setups above 0.72 (Table 1). One source of inaccuracy seems to be that GPU kernels can have varying performance depending on input shapes even when the floating point operations are identical—modeling this should improve accuracy. Also, the point of the cost model is to select the most inexpensive plan, and we note that the selected plan was never beaten by any sampled alternative on the transformer block, indicating that the proxy still ranks plans reasonably.

Finally, a note on the choice of baselines. We compare against expertly-tuned production systems rather than against prior auto-parallelizing research systems. Alpha [42] can no longer be built against a current JAX stack, and the systems that remain maintained, Unity [35] and nnScaler [25], generate plans for training rather than for the inference workloads studied here.

7 RELATED WORK

Automatic parallelization of AI computations. The named intra-operator strategies in wide use, such as tensor (model) parallelism [32], sharded data parallelism [28], and sequence parallelism [22], were each designed by hand. Auto-parallelizing systems search over combinations of such strategies: FlexFlow [20] searches per-operator parallelization configurations with a randomized strategy, GSPMD [39] is a partitioning *mechanism* that propagates user-supplied sharding annotations through a computation graph and Alpha [42] automates both inter-operator (pipeline) and intra-operator parallelism, the latter via an integer program over per-operator shardings of tensor dimensions onto a two-dimensional logical device mesh. Unity [35] extends FlexFlow to jointly optimize algebraic rewrites and parallelization for training. nnScaler [25]

goes furthest in our direction, letting each operator declare its partitionable dimensions with an einsum-like annotation; partitioning remains uniform per dimension, aggregation is fixed to summation, and communication is pattern-matched onto NCCL collectives. Einsummable’s join-agg specs are instead produced by operator code, admitting non-uniform, data-dependent decompositions and arbitrary $\oplus$, and its communication is synthesized for the topology.

Collective communication. A recent line of work synthesizes topology-aware *implementations* of the collectives themselves: SCCL [5] and TACCL [31] search for optimal algorithms realizing a given collective on a given interconnect, and Blink [37] builds collectives from packing spanning trees on heterogeneous topologies. Einsummable takes the further step of having no fixed communication primitives at all. Each exchange program is a special-purpose communication-and-aggregation pattern.

Relations on tensor runtimes. Some systems run relational workloads on tensor infrastructure (TQP [14], Hummingbird [26], Koutsoukos et al. [23]) or bring linear algebra and even neural-network training into relational engines (SystemML [11], SystemDS [2], Schüle et al. [30]); Einsummable works in the other direction, using relational optimization for tensor workloads.

Tensor-relational optimization. Einsummable builds directly on the tensor-relational model [41], which represents tensors as relations of (key, sub-tensor) pairs so that kernels do enough work to run efficiently on accelerators, and on the broader argument that relational systems are a sound substrate for distributed ML [18, 19, 33]. Our earlier EinDecomp system [3] takes a first step, selecting decompositions with a cost-based dynamic program, but only for computations expressed in an extended Einstein-summation notation (ruling out FlashAttention, for example). SPORES [38] optimizes linear-algebra expressions via relational equality saturation, STOREL [29] optimizes a tensor program given user-supplied storage formats, and Galley [8] brings cost-based query optimization to sparse tensor programs. Unlike Einsummable, these target logical rewrites or single-node kernel structure.

Kernel compilers. TVM [6], Triton [34], and TACO [21] generate high-performance kernels for tensor operations on a single device. This work is complementary. Einsummable consumes kernels (cuBLAS matrix multiplies, FlashAttention [7]) as well as kernels produced by these compilers as its concrete operators.

Parallel database systems. Intra-operator, partitioned parallelism with automatic orchestration is the founding idea of shared-nothing parallel database systems [10] and of Volcano’s exchange operator [12], which exchange programs generalize (Section 5).

8 CONCLUSIONS

We have presented Einsummable, which automatically distributes an AI computation across the GPUs of a server by treating every operation as a relational join followed by an aggregation over tensor relations. Operations export their decompositions as join-agg specs, which are selected to minimize communication, and exchange programs synthesized against the hardware topology realize the plan without canned collectives.

There is much work to do. Extending the logical optimization phase so that it is aware of computation cost (and of device properties and how they interact with computational efficiency) would increase accuracy. Extending exchange programs to allow for multi-machine parallelism as well as incorporating learning (to understand how the simulator differs from reality) will be crucial.

REFERENCES

- [1] Ramesh C Agarwal, Susanne M Balle, Fred G Gustavson, Mahesh Joshi, and Prasad Palkar. 1995. A three-dimensional approach to parallel matrix multiplication. *IBM Journal of Research and Development* 39, 5 (1995), 575–582.
- [2] Matthias Boehm, Iulian Antonov, Sebastian Baunsgaard, Mark Dokter, Robert Ginhör, Kevin Innerebner, Florian Klezin, Stefanie Lindstaedt, Arnab Phani, Benjamin Rath, Berthold Reinwald, Shafaq Siddiqui, and Sebastian Benjamin Wrede. 2020. SystemDS: A Declarative Machine Learning System for the End-to-End Data Science Lifecycle. In *10th Conference on Innovative Data Systems Research (CIDR)*.
- [3] Daniel Bourgeois, Zhimin Ding, Dimitrije Jankov, Jiehui Li, Sleem Mahmoud Abdelghafar, Yuxin Tang, Jiawen Yao, Xinyu Yao, and Chris Jermaine. 2025. EinDecomp: Decomposition of Declaratively-Specified Machine Learning and Numerical Computations for Parallel Execution. *Proc. VLDB Endow.* 18, 7 (2025), 2240–2253. <https://doi.org/10.14778/3734839.3734858>
- [4] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Nécule, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, et al. 2018. JAX: composable transformations of Python+NumPy programs. <http://github.com/google/jax>.
- [5] Zixian Cai, Zhengyang Liu, Saeed Maleki, Madanlal Musuvathi, Todd Mytkowicz, Jacob Nelson, and Olli Saarikivi. 2021. Synthesizing optimal collective algorithms. In *Proceedings of the 26th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 62–75.
- [6] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Lianmin Zheng, Eddie Yan, Haichen Shen, Meghan Cowan, Leyuan Wang, Yuwei Hu, Luis Ceze, et al. 2018. {TVM}: An automated {End-to-End} optimizing compiler for deep learning. In *13th USENIX symposium on operating systems design and implementation (OSDI 18)*, 578–594.
- [7] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in neural information processing systems* 35 (2022), 16344–16359.
- [8] Kyle Deeds, Willow Ahrens, Magdalena Balazinska, and Dan Suciu. 2025. Galley: Modern query optimization for sparse tensor programs. *Proceedings of the ACM on Management of Data* 3, 3 (2025), 1–24.
- [9] Eliezer Dekel, David Nassimi, and Sartaj Sahni. 1981. Parallel matrix and graph algorithms. *SIAM Journal on computing* 10, 4 (1981), 657–675.
- [10] David DeWitt and Jim Gray. 1992. Parallel database systems: The future of high performance database systems. *Commun. ACM* 35, 6 (1992), 85–98.
- [11] Amol Ghoting, Rajasekar Krishnamurthy, Edwin Pednault, Berthold Reinwald, Vikas Sindhwani, Shirish Tatikonda, Yuanyuan Tian, and Shivakumar Vaithyanathan. 2011. SystemML: Declarative machine learning on MapReduce. In *2011 IEEE 27th International conference on data engineering*. IEEE, 231–242.
- [12] Goetz Graefe. 1990. Encapsulation of parallelism in the volcano query processing system. *ACM SIGMOD Record* 19, 2 (1990), 102–111.
- [13] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [14] Dong He, Supun Nakandala, Dalitso Banda, Rathijit Sen, Karla Saur, Kwanghyun Park, Carlo Curino, Jesús Camacho-Rodríguez, Konstantinos Karanasos, and Matteo Interlandi. 2022. Query Processing on Tensor Computation Runtimes. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2811–2825.
- [15] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, Hyukjoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [16] Dror Irony, Sivan Toledo, and Alexander Tiskin. 2004. Communication lower bounds for distributed-memory matrix multiplication. *J. Parallel and Distrib. Comput.* 64, 9 (2004), 1017–1026.
- [17] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. DeepSpeed ullyses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509* (2023).
- [18] Dimitrije Jankov, Shangyu Luo, Binhang Yuan, Zhuhua Cai, Jia Zou, Chris Jermaine, and Zekai J. Gao. 2019. Declarative Recursive Computation on an RDBMS: or, Why You Should Use a Database for Distributed Machine Learning. *Proceedings of the VLDB Endowment* 12, 7 (2019), 822–835.
- [19] Dimitrije Jankov, Binhang Yuan, Shangyu Luo, and Chris Jermaine. 2021. Distributed Numerical and Machine Learning Computations via Two-Phase Execution of Aggregated Join Trees. *Proceedings of the VLDB Endowment* 14, 7 (2021), 1228–1240.
- [20] Zhihao Jia, Matei Zaharia, and Alex Aiken. 2019. Beyond data and model parallelism for deep neural networks. *Proceedings of Machine Learning and Systems* 1 (2019), 1–13.
- [21] Fredrik Kjolstad, Shoaib Kamil, Stephen Chou, David Lugato, and Saman Amarasinghe. 2017. The tensor algebra compiler. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–29.
- [22] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* 5 (2023), 341–353.
- [23] Dimitrios Koutsoukos, Supun Nakandala, Konstantinos Karanasos, Karla Saur, Gustavo Alonso, and Matteo Interlandi. 2021. Tensors: An abstraction for general data processing. *Proceedings of the VLDB Endowment* 14, 10 (2021), 1797–1804.
- [24] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th symposium on operating systems principles*, 611–626.
- [25] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, Cheng Li, Saeed Maleki, Xu Cao, Ning Shang, Yilei Yang, et al. 2024. {nnScaler}::Constrained-Guided parallelization plan generation for deep learning training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 347–363.
- [26] Supun Nakandala, Karla Saur, Gyeong-In Yu, Konstantinos Karanasos, Carlo Curino, Markus Weimer, and Matteo Interlandi. 2020. A Tensor Compiler for Unified Machine Learning Prediction Serving. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*.
- [27] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [28] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *SC20: international conference for high performance computing, networking, storage and analysis*. IEEE, 1–16.
- [29] Maximilian Schleich, Amir Shaikhha, and Dan Suciu. 2023. Optimizing tensor programs on flexible storage. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27.
- [30] Maximilian E. Schüle, Thomas Neumann, and Alfons Kemper. 2024. The Duck’s Brain: Training and Inference of Neural Networks in Modern Database Engines. *Datenbank-Spektrum* 24, 3 (2024).
- [31] Aashaka Shah, Vijay Chidambaram, Meghan Cowan, Saeed Maleki, Madan Musuvathi, Todd Mytkowicz, Jacob Nelson, Olli Saarikivi, and Rachee Singh. 2023. {TACCL}::Guiding collective algorithm synthesis using communication sketches. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, 593–612.
- [32] Mohammad Shoeybi, Mostafa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [33] Yuxin Tang, Zhimin Ding, Dimitrije Jankov, Binhang Yuan, Daniel Bourgeois, and Chris Jermaine. 2023. Auto-Differentiation of Relational Computations for Very Large Scale Machine Learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*.
- [34] Philippe Tillet, Hsiang-Tsung Kung, and David Cox. 2019. Triton: an intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 10–19.
- [35] Colin Unger, Zhihao Jia, Wei Wu, Sina Lin, Mandeep Baines, Carlos Efrain Quintero Narvaez, Vinay Ramakrishnaiah, Nirmal Prajapati, Patrick S. McCormick, Jamaludin Mohd-Yusof, Xi Luo, Dheevatsa Mudigere, Jongsoo Park, Misha Smelyanskiy, and Alex Aiken. 2022. Unity: Accelerating DNN Training Through Joint Optimization of Algebraic Transformations and Parallelization. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 267–284.
- [36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [37] Guanhua Wang, Shivaram Venkataraman, Amar Phanishayee, Nikhil Devanur, Jorgen Thelin, and Ion Stoica. 2020. Blink: Fast and generic collectives for distributed ml. *Proceedings of Machine Learning and Systems* 2 (2020), 172–186.
- [38] Yisu Remy Wang, Shana Hutchison, Jonathan Leang, Bill Howe, and Dan Suciu. 2020. SPORES: Sum-Product Optimization via Relational Equality Saturation for Large Scale Linear Algebra. *Proceedings of the VLDB Endowment* 13, 12 (2020), 1919–1932.
- [39] Yuanzhong Xu, Hyukjoong Lee, Dehao Chen, Blake Hechtman, Yanping Huang, Rahul Joshi, Maxim Krikun, Dmitry Lepikhin, Andy Ly, Marcello Maggioni,

- Ruoming Pang, Noam Shazeer, Shibo Wang, Tao Wang, Yonghui Wu, and Zhifeng Chen. 2021. GSPMD: General and Scalable Parallelization for ML Computation Graphs. *arXiv preprint arXiv:2105.04663* (2021).
- [40] Weipeng P. Yan and Per-Åke Larson. 1995. Eager Aggregation and Lazy Aggregation. In *Proceedings of the 21st International Conference on Very Large Data Bases (VLDB)*. 345–357.
- [41] Binhang Yuan, Dimitrije Jankov, Jia Zou, Yuxin Tang, Daniel Bourgeois, and Chris Jermaine. 2021. Tensor relational algebra for distributed machine learning system design. *Proceedings of the VLDB Endowment* 14, 8 (2021).
- [42] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 559–578.